Developer quickstart

Drishti · docs/QUICKSTART_DEVELOPERS.md · Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

From a clean machine to your first change, built, checked and committed, in about an hour (most of it the first build and the demo lake). This page is the shortest path; each step links the guide that holds the details, so nothing here is the only place a fact lives: DEVELOPER_GUIDE.md (the code, the rules, recipes), BUILD_GUIDE.md (packs and screens), CLI_REFERENCE.md (every command), CONFIGURATION.md (every setting). To install and verify a release rather than develop it, use DEPLOYMENT_AND_QA_GUIDE.md. A PDF copy of this page is QUICKSTART_DEVELOPERS.pdf.

Contents

1. Clone and set up the machine
2. Build the server
3. Run the server and the console on the demo lake
4. The repository map
5. The fast checks
6. The rules a change must follow
7. Your first change, end to end
8. Where to go next

1. Clone and set up the machine

You need a JDK 21 (production runs 21; 25 is also tested), Python 3.13 for the console and the tools, git, and Node 20 or newer for the Elements unit tests. Maven is not needed: the repository carries its own (`./mvnw`, `mvnw.cmd`). The one statement of the versions is OPERATIONS_GUIDE.md › Prerequisites.

	Linux (Ubuntu, Debian)	**macOS**	**Windows (PowerShell)**
JDK 21	`sudo apt install openjdk-21-jdk`	`brew install openjdk@21`	the Temurin 21 MSI or ZIP (a ZIP needs no installer)
Python 3.13	`uv python install 3.13`, or the system's	`brew install python@3.13`, or uv	python.org's installer or its embeddable ZIP
uv (recommended)	the installer script from astral.sh (`install.sh`)	`brew install uv`	optional: `py -3.13 -m venv` works without it
Node (Elements tests)	`sudo apt install nodejs`	`brew install node`	optional: the Node tests skip without it
`JAVA_HOME`	`export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64`	`export JAVA_HOME=$(/usr/libexec/java_home -v 21)`	`$env:JAVA_HOME = 'C:\jdk-21'`

```
git clone https://github.com/ajsinha/drishti.git
cd drishti                                    # every command on this page runs from here
uv venv --python 3.13 drishti-console/.venv
uv pip install --python drishti-console/.venv/bin/python -r drishti-console/requirements-test.txt
drishti-console/.venv/bin/python -m playwright install chromium        # the browser tests' Chromium, once
drishti-console/.venv/bin/python tools/drishti.py doctor
```

On Windows the environment's Python is `drishti-console\.venv\Scripts\python.exe` (`py -3.13 -m venv drishti-console\.venv`, then `drishti-console\.venv\Scripts\python.exe -m pip install -r drishti-console\requirements-test.txt`).

You should see `doctor` end with `doctor: all clear` (the Java, the Python and its libraries, the ports 18480 and 17480 free). `requirements-test.txt` is `requirements.txt` plus pytest, pytest-xdist and Playwright: a developer needs both. The IDE setup (IntelliJ IDEA on the root, PyCharm on `drishti-console/`, never one folder for both) is OPERATIONS_GUIDE.md › Maven, IntelliJ IDEA and PyCharm.

---

2. Build the server

```
./mvnw -q clean package -DskipTests -pl drishti-app -am
ls drishti-server/target/*-exec.jar
```

You should see nothing from Maven (`-q`), then `drishti-server/target/drishti-server-2.9.11.g-exec.jar`. The first build downloads its libraries (several minutes); later ones take about a minute. `-pl drishti-app -am` builds the jar that runs (the core server with the banking module and the plugins) and only what it needs; `clean` keeps exactly one exec jar in `target/`. Every Maven command and what it builds: DEVELOPER_GUIDE.md › Maven commands. On Windows: `.\mvnw.cmd -q clean package -DskipTests -pl drishti-app -am`.

---

3. Run the server and the console on the demo lake

Build the demo lake once (86 tables, sixty business days ending today, about 15 minutes and 3.4 GB under `data/delta/demo/`), then start both programs with the scripts in `bin/`:

```
drishti-console/.venv/bin/python tools/drishti.py lake demo
```

Linux and macOS

```
bin/start-all.sh --demo          # the server on :18480, then the console on :17480, in the backgroun
d
bin/status.sh                    # both running and UP, with the addresses to open
bin/stop-all.sh                  # the console, then the server (by their pid files)
```

Windows (`cmd` or PowerShell; Java and Python only)

```
bin\start-all.cmd --demo
bin\status.cmd
bin\stop-all.cmd
```

You should see `server: up on http://<this machine's address>:18480` and `console: up on http://<this machine's address>:17480`, with a warning that the demo's **development** secrets are in use (fine on your machine, never on a shared one). Open the console's address, sign in as `drishti-dev-admin` (password `drishti-dev-admin123`), type `TRD 1000001` and press Enter: an ENDUR commodity swap with its legs and cash flows. `VAR BANK` answers the bank's value at risk.

To watch a program's output, start each in the foreground in its own terminal instead (`Ctrl+C` stops it): `bin/start-server.sh --demo`, then `bin/start-console.sh --demo` (Windows: `bin\start-server.cmd --demo`, `bin\start-console.cmd --demo`). Logs are in `data/logs/` (`server.log`, `console.log`), process ids in `data/run/`. Every option of the scripts (ports, heap, an env-file for your own secrets, a cluster of several servers with `--node`, the console's `--server`), where the files go, the exit codes and what to do when a script stops with an error: DEPLOYMENT_AND_QA_GUIDE.md › section 6. The IntelliJ run configurations **Drishti server (demo lakes)** and **Drishti console** set the same demo settings for debugging; why each setting: OPERATIONS_GUIDE.md › Start the server and › Start the console (docs/guides/OPERATIONS_GUIDE.md#26-start-the-console).

Everything the two write at run time is under `data/` (git-ignored except `data/testdata/`); deleting `data/delta`, `data/bi-store` and `data/cache` while both are stopped starts you fresh.

Starting a white-labelled instance

The same build starts as **Drishti Risk**, **Drishti Finance**, **Drishti Genomics** or **Drishti All** with one argument (`config/distributions/drishti.<name>.yaml`; DISTRIBUTIONS.md). Without it you run Drishti All, every module: what the drill, the page walk and the tutorials use.

Distribution	Linux and macOS	Windows	You should see
Drishti Risk	`bin/start-all.sh --demo --distribution risk`	`bin\start-all.cmd --demo --distribution risk`	**Drishti Risk**, navy dark; Risk, Counterparties and Markets menus; `VAR BANK` works, `ECON CAN` answers *not in Drishti Risk*; Help titled *Drishti Risk Help*
Drishti Finance	`bin/start-all.sh --demo --distribution finance`	`bin\start-all.cmd --demo --distribution finance`	**Drishti Finance**, light; Economies, Markets and Companies menus, no Risk menu; `CRV` works, `VAR BANK` answers *not in Drishti Finance*; Help titled *Drishti Finance Help* with no VaR guide
Drishti Genomics	`bin/start-all.sh --demo --distribution genomics`	`bin\start-all.cmd --demo --distribution genomics`	**Drishti Genomics**, green; the kernel's menus only; Admin → Health: module `genomics` *not installed* (the sample ships later)
Drishti All	`bin/start-all.sh --demo`	`bin\start-all.cmd --demo`	**Drishti**, exactly as above

`bin/status.sh` (Windows `bin\status.cmd`) shows each process's distribution; Admin → White label shows them all side by side. The setting behind the argument is `DRISHTI_DISTRIBUTION` (both programs read it).

In the IDEs. IntelliJ IDEA lists, under *Drishti*, `Drishti server (Risk)`, `(Finance)` and `(All)` (each sets the environment variable `DRISHTI_DISTRIBUTION`) and the Python configurations `Drishti console (Risk)` (argument `--distribution=risk`, port 17480), `Drishti console (Finance)` (`--distribution=finance`, port 17481) and `Drishti console (All)` (`--distribution=all`), all in `run-configs/`. Run the console ones from IntelliJ with the Python plugin or from PyCharm opened on the repository root. **Two consoles against one server:** run `Drishti server (All)`, then `Drishti console (Risk)` and `Drishti console (Finance)`; Drishti Risk opens on `http://localhost:17480` and Drishti Finance on `http://127.0.0.1:17481` (use a different host name, or another browser profile, for the second: browsers share sign-in cookies across ports of one host). Every step, Linux, macOS and Windows: DEPLOYMENT_AND_QA_GUIDE.md › 6.7.

4. The repository map

Two programs: the **server** (Java 21, Spring Boot, a Maven reactor) computes every number and lays out every screen; the **console** (Python, FastAPI and Jinja2) renders what the server sends, and the browser draws it. Domains are **modules**: the core plus banking, capitalmarkets and economy. The full map, module by module: DEVELOPER_GUIDE.md › The repository.

Where	What
`drishti-api`, `drishti-common`	the source SPI (Spring-free) and the shared types, errors and file-date rules
`drishti-engine`, `drishti-rachana`, `drishti-inference`, `drishti-graph`	the view pipeline, the Rachana grammar (Sutras, the expression language), layout inference, links
`drishti-registry`	modules, lenses, functions and their switches: what is on at this site
`drishti-banking`, `drishti-compute-banking`	the banking module: VaR and ES, exposure, FRTB-SA, SIMM, sensitivities, the MR cube; its compute kernels
`drishti-lakeconfig`, `drishti-lakeio`, `drishti-deltalake`, `drishti-iceberg`, `drishti-memlake`, `drishti-lakeload`, `drishti-dataload`	the lakes: their files, readers and writers, the in-memory lake, the parallel loader and the data inbox
`drishti-identity`, `drishti-net`, `drishti-compute`, `drishti-bicompute`, `drishti-tables`, `drishti-flight`, `drishti-ai`	users and tokens; Drishti*net (clusters); compute workers; the pivot and BI engine; tables and models; Arrow Flight; the AI layer
`drishti-server`	the Spring Boot application: controllers, the core's configuration, the exec jar (`drishti-server/target/*-exec.jar`)
`drishti-app`	the assembly the release ships: the server with the banking module and the plugins (`-pl drishti-app -am`)
`plugins/drishti-plugin-*`	the connectors: delta, iceberg, file, jdbc, kafka, s3, rest, feeds and more (DATA_GUIDE.md)
`drishti-elements`, `drishti-console/web/static/elements/`	Drishti Elements, the JavaScript libraries every screen is made of (core, inputs, grid, chart, data, insight, renderer, risk, graph, tree, designer), and their WebJar (ELEMENTS_API.md)
`drishti-console/`	`run_drishti_web.py`, `core/` (one module per feature), `routes/`, `web/templates/`, `plugins/<module>/` (a module's console pages, help and static files: `banking`, `economy`), `config/` (`application.yaml`, `help.yaml`), `tests/`
`config/packs/<domain>/<pack>/`	the packs (content, not code): `core/`, `banking/`, `capitalmarkets/`, `economy/` (BUILD_GUIDE.md)
`config/` (the rest)	`modules/`, `lenses/<module>/`, `lakes/`, `engines/`, `tables/`, `bi/`, `feeds/`: the site's switches and files (CONFIGURATION.md)
`bin/`	the start and stop scripts of the server, its cluster nodes and the console (`.sh` for Linux and macOS, `.ps1` and `.cmd` for Windows; bin/README.md)
`tools/`	`drishti.py` (the command line), `lake/`, `docs/` (screenshots, the PDFs), `drill.sh` (the release drill), `windows/` (PowerShell loaders and feeds)

Where	What
`docs/`	`guides/`, `tutorials/`, `admin/`, `architecture/`, `contracts/`; the index is docs/README.md
`drishti-it`, `drishti-benchmarks`	the architecture and repository rule tests; the JMH benchmarks

5. The fast checks

Run these before every commit; each takes seconds to a few minutes. The whole test strategy is DEVELOPER_GUIDE.md › Testing strategy.

Check	Command	What it covers
The quick file checks	`drishti-console/.venv/bin/python -m pytest -q -m static drishti-console/tests`	every test that reads files and starts nothing: docs and help, asset names, branding, CSS tokens, guards, licences
The Elements unit tests	`node --test drishti-console/tests/elements/` (or `pytest drishti-console/tests/test_elements_node.py`)	the libraries in Node, no browser
One browser file	`drishti-console/.venv/bin/python -m pytest -q drishti-console/tests/test_elements_gallery_browser.py`	Chromium on a real console; name the file of the screen you changed
One Java module	`./mvnw -q verify -pl drishti-registry -am`	that module's tests and the rules it must keep
The gate	`tools/drill.sh --gate`	everything the release drill checks first (headers, generated files, docs, static tests, the rule tests and one build); a few minutes, on any branch, no push

You should see pytest end with `passed` and no `failed`, and `drill.sh --gate` end with its summary and exit 0. Read the exit code before trimming a long output: `… ; echo "exit=$?"`. The gate is the one check the release drill runs first; run it before you hand a branch over. Never run two heavy jobs (a gate, a drill, a full verify) at once on one machine.

6. The rules a change must follow

Most rules are tests, so the fast checks above catch them; the rest is review. The full list with the test of each: DEVELOPER_GUIDE.md › Project rules.

Rule	Held by
No bespoke UI: every screen, designer and editor is built from Drishti Elements; a missing piece is added to a library, never written for one page	`test_elements_guards.py`, `test_designer_kit_guard.py`, `test_admin_forms_guard.py`, `test_native_select_guard.py`
CSS tokens: colours, spacing and type come from the tokens, never literals	`test_css_tokens.py`
Domain boundaries: the core never imports a module; banking code lives in the banking module and its console plugin	`test_module_boundaries.py`, `ArchitectureRulesTest`

Rule	Held by
The servers compute, the browser draws: every risk and BI number is computed in Java on the server	review; ARCHITECTURE.md
Branding from configuration: no product name or owner in code or templates (`ui.product`, `drishti.branding.*`)	`test_branding_guard.py`
Our files are named `drishti_<name>` (JS, CSS, images we write); vendored libraries untouched under `static/vendor` with `VENDOR.md`; nothing from a CDN	`test_assets_policy.py` and the asset-name tests
The copyright header on every source file; no source file over 2,000 lines	`LicenseHeaderTest`, `tools/license_headers.py`, `SourceFileSizeTest`
Docs with every change: the guide, tutorial, Help entry and CHANGELOG that a change affects are updated in the same commit; one home per fact (link, do not copy); no bugs or version history in customer documents	`test_docs_rot.py`, `test_docs_gates.py`, `test_help*.py`, `test_guide_images.py`
Settings: a new setting is declared where the generator finds it and appears in CONFIGURATION.md	the generated-document checks of the gate
Commits: authored as the repository owner, plain sentences that say what changed for a user or an operator, no tool or co-author trailers	review (DEVELOPER_GUIDE.md › Git)

7. Your first change, end to end

Every change takes the same steps, however small; the example is a change to how a grid behaves on one screen, which touches a library, a test, a guide and a screenshot together.

1. **Branch** from `develop`: `git switch -c my-first-change origin/develop`.
2. **Find the one home.** Search before writing: `git grep -n "the thing you change"` across `drishti-console/`, `config/` and `docs/`. A screen is made of Elements: change the library (`drishti-console/web/static/elements/<library>/`) or the page's template, never a one-off widget (ELEMENTS_API.md). A pack change is YAML under `config/packs/` (BUILD_GUIDE.md). A Java change follows the recipes of DEVELOPER_GUIDE.md › Recipes (a REST endpoint, a configuration property, a console page, a source plugin).
3. **Write the test first** beside its neighbours: a `drishti-console/tests/test_<area>.py` for the console (a `*_browser.py` when it needs Chromium), a JUnit test in the module for Java, a `tests/elements/<lib>.test.mjs` for a library.
4. **Make the change**, then run the server and the console (section 3) and look at it: the page, at desktop width and at phone width (390 px, the browser's device toolbar).
5. **Update the docs in the same commit**: the guide section that describes the behaviour, the tutorial step if a step changes, the Help entry (`drishti-console/config/help.yaml`) if a page is added or moves, the screenshots of the step (`drishti-console/.venv/bin/python tools/docs/screenshots.py --guide <guide>`), and one line under the open release in `CHANGELOG.md`. A setting goes into CONFIGURATION.md through its generator, never by hand.
6. **Run the fast checks** (section 5): `-m static`, the test files you touched, the Elements tests for a library change, then `tools/drill.sh --gate`.
7. **Commit** as the owner, one plain sentence of what changed for whom: `git -c user.name=ajsinha -c user.email=ajsinha@gmail.com commit -am "Pivot: the totals row stays visible while the grid scrolls"`.
8. **Push the branch** and hand it over; `main` moves only through the release drill (DEVELOPER_GUIDE.md › The drill).

You should see the gate pass on your branch, the docs checks green (no broken link, no missing picture, Help listing the page) and your change in the running console.

8. Where to go next

You want to	Read
understand how a request travels through the code	DEVELOPER_GUIDE.md › How a request travels
add a pack, a kind, a Sutra or a screen	BUILD_GUIDE.md and tutorial 19 (docs/tutorials/19-build-a-pack-and-a-screen.md)
add or change a connector	DEVELOPER_GUIDE.md › Connectors
use or extend the Elements libraries	ELEMENTS_API.md
every command of `drishti.py`	CLI_REFERENCE.md
every setting	CONFIGURATION.md
install, deploy and verify a release on Windows, Linux or macOS	DEPLOYMENT_AND_QA_GUIDE.md
the browser test harness	TEST_HARNESS.md

Copyright (c) 2026 Ashutosh Sinha ajsinha@gmail.com (docs/mailto:ajsinha@gmail.com). All rights reserved. Proprietary and confidential. Unauthorised copying, use or distribution is prohibited.